204252 Data Structures and Analysis

Chapter 9: Sets, Maps, and Dictionaries (Part 2)

Thapanapong Rukkanchanunt and Jakarin Chawachat

August 13, 2026

Computer Science Department

Outline

List-Based Dictionary Implementations

Tree-Based Dictionary Implementations

Hash-Based Dictionary Implementations

Summary of Dictionary Architectures

List-Based Dictionary Implementations

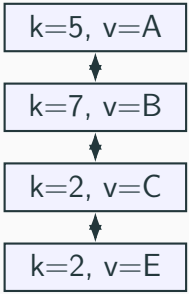
1. Unordered List (Log File)

The simplest dictionary representation: elements are appended to a doubly-linked list or array in arbitrary order.

Operation Cost

- `insert(k, v)`: $\mathcal{O}(1)$ (just append).
- `find(k)`: $\mathcal{O}(N)$ (linear scan).
- `erase(k)`: $\mathcal{O}(N)$ (scan, then delete).

Log File Memory Layout

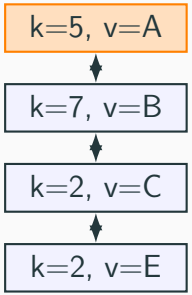


Log File Erase Trace: Step-by-Step

We want to execute `erase(2)` on a Log File containing: `{ (5,A), (7,B), (2,C), (2,E) }`.

Step-by-step execution:

- **Initial state**: We start at the head of the list.

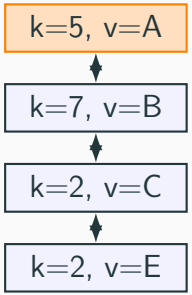


Log File Erase Trace: Step-by-Step

We want to execute `erase(2)` on a Log File containing: `{ (5,A), (7,B), (2,C), (2,E) }`.

Step-by-step execution:

- **Step 1**: Compare key 5 with target 2. No match! Move to next.

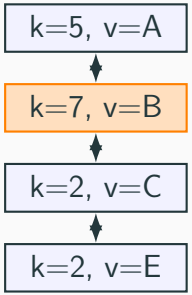


Log File Erase Trace: Step-by-Step

We want to execute `erase(2)` on a Log File containing: `{ (5,A), (7,B), (2,C), (2,E) }`.

Step-by-step execution:

- **Step 2**: Compare key 7 with target 2. No match! Move to next.

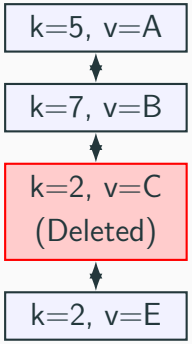


Log File Erase Trace: Step-by-Step

We want to execute `erase(2)` on a Log File containing: `{ (5,A), (7,B), (2,C), (2,E) }`.

Step-by-step execution:

- **Step 3**: Compare key 2 with target 2. Match! Delete node `(2, C)`.

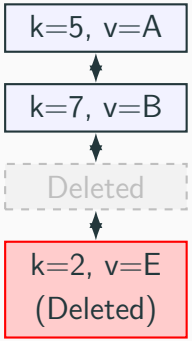


Log File Erase Trace: Step-by-Step

We want to execute `erase(2)` on a Log File containing: `{ (5,A), (7,B), (2,C), (2,E) }`.

Step-by-step execution:

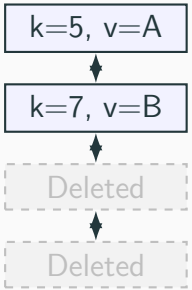
- **Step 4**: Move to next. Compare key 2 with target 2. Match! Delete node `(2, E)`.



Log File Erase Trace: Step-by-Step

We want to execute `erase(2)` on a Log File containing: `{ (5,A), (7,B), (2,C), (2,E) }`.

Step-by-step execution:



- **Final state**: Scan complete. We removed all instances of key 2.

2. Ordered Array / Sequence

What if we store the items in an array sorted by key?

Operation Cost

- `find(k)`: $\mathcal{O}(\log N)$ using **Binary Search**.
- `insert(k, v)`: $\mathcal{O}(N)$ (must shift elements to insert in order).
- `erase(k)`: $\mathcal{O}(N)$ (must shift elements to fill the gap).

What if we use a sorted Linked List?
Binary search is **impossible** in $\mathcal{O}(\log N)$ time on a linked list due to sequential access ($\mathcal{O}(N)$ to find center). Thus, find degrades to $\mathcal{O}(N)$.

Ordered Array Layout

k=2 v=C	k=2 v=E	k=5 v=A	k=7 v=B
idx 0	idx 1	idx 2	idx 3

Binary Search (`find(5)`):

- Midpoint (idx 1) is key $2 < 5$.
- Search right side.
- Locate key 5 at idx 2 in $\mathcal{O}(\log N)$ time.

Tree-Based Dictionary Implementations

Tree-Based Dictionaries

Tree-based dictionaries organize keys in hierarchical structures.

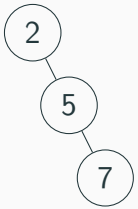
Binary Search Tree (BST)

- Average case: $\mathcal{O}(\log N)$ for find, insert, delete.
- Worst case: $\mathcal{O}(N)$ if keys are inserted in sorted order (linear tree).

Balanced Trees (AVL, Red-Black)

- Worst-case: $\mathcal{O}(\log N)$ guaranteed.
- Rotations keep tree height balanced.
- Used in `std::multimap` in C++.

Degenerate BST (Worst Case)

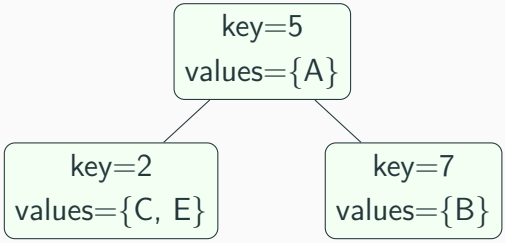


Handling Duplicates in Tree-Based Structures

Since dictionaries allow duplicate keys, how do we store them in a tree?

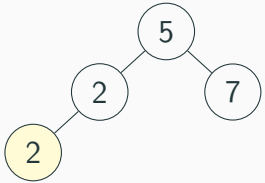
Approach A: Separate Chaining in Nodes

- Each node holds a unique key and a list of values (e.g. `std::vector<Value>`).
- Keeps tree smaller, rotations are identical to standard maps.



Approach B: Multiple Nodes with Same Key

- Define BST property: left child $\leq$ parent.
- Duplicate keys are inserted as independent nodes.
- Increases tree height; traversal must continue scanning subtrees.



BST Duplicate Insertion Trace: Step-by-Step

Inserting keys in order: `5, 2, 7, 2` (duplicate) using the property `left` $\leq$ `parent`.

Step-by-step insertion:

- **Step 1**: Insert 5. Tree is empty, so it becomes the root.

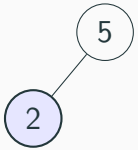


BST Duplicate Insertion Trace: Step-by-Step

Inserting keys in order: `5, 2, 7, 2` (duplicate) using the property `left` $\leq$ `parent`.

Step-by-step insertion:

- **Step 2**: Insert 2. Compare with 5: $2 \leq 5$, go left. Left child is empty, insert 2.

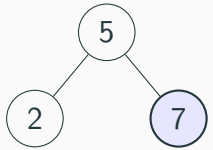


BST Duplicate Insertion Trace: Step-by-Step

Inserting keys in order: `5, 2, 7, 2` (duplicate) using the property `left` $\leq$ `parent`.

Step-by-step insertion:

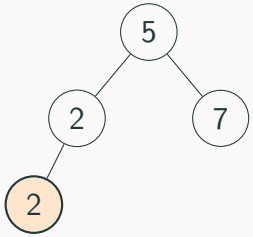
- **Step 3**: Insert 7. Compare with 5: $7 > 5$, go right. Right child is empty, insert 7.



BST Duplicate Insertion Trace: Step-by-Step

Inserting keys in order: `5, 2, 7, 2` (duplicate) using the property `left` $\leq$ `parent`.

Step-by-step insertion:



- **Step 4**: Insert 2 (duplicate).
  - Compare with root 5: $2 \leq 5$, go left.
  - Compare with node 2: $2 \leq 2$, go left!
  - Left child of 2 is empty, insert duplicate 2.

Hash-Based Dictionary Implementations

Hash Table Dictionary Basics

A **Hash Table** uses a hash function $h(K)$ to map keys to array slots (buckets).

- **Average Performance**: $\mathcal{O}(1)$ insertion, lookup, and deletion.
- **Worst Performance**: $\mathcal{O}(N)$ if all elements hash to the same bucket.

The Collision Problem

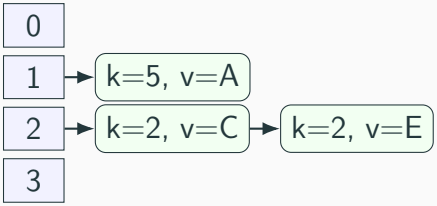
Since the key space is much larger than the table size M, multiple keys will inevitably hash to the same slot ($h(K_1) = h(K_2)$). We need collision resolution!

Collision Resolution 1: Separate Chaining

Each bucket of the hash table is a pointer to a linked list of entries that hashed to that slot.

- **Duplicates**: Handled easily by appending duplicate key-value pairs to the same chain.
- **Load Factor ($\lambda = N/M$)**: Can be > 1. Average search time is $\mathcal{O}(1 + \lambda)$.

Separate Chaining Diagram



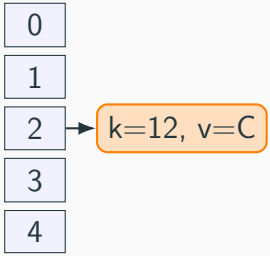
Separate Chaining Dictionary Trace: Step-by-Step

Inserting items: `(12, C)`, `(7, B)`, `(12, E)` into a hash table of size $M = 5$.

Hash function: $h(K) = K \bmod 5$.

Step-by-step insertion:

- **Step 1**: Insert `(12, C)`.
  $h(12) = 12 \bmod 5 = 2$.
  Append to chain at slot 2.



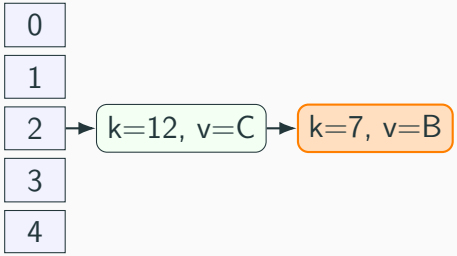
Separate Chaining Dictionary Trace: Step-by-Step

Inserting items: `(12, C)`, `(7, B)`, `(12, E)` into a hash table of size $M = 5$.

Hash function: $h(K) = K \bmod 5$.

Step-by-step insertion:

- **Step 2**: Insert `(7, B)`.
  $h(7) = 7 \bmod 5 = 2$.
  Append to chain at slot 2. Chain is now: `(12, C)` $\rightarrow$ `(7, B)`.



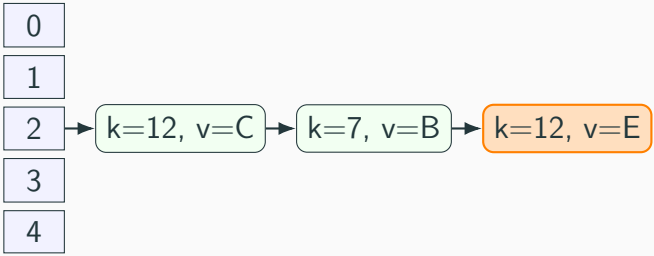
Separate Chaining Dictionary Trace: Step-by-Step

Inserting items: `(12, C)`, `(7, B)`, `(12, E)` into a hash table of size $M = 5$.

Hash function: $h(K) = K \bmod 5$.

Step-by-step insertion:

- **Step 3**: Insert `(12, E)` (duplicate key!).
  $h(12) = 2$.
  Append to chain at slot 2. Chain is now: `(12, C)` $\rightarrow$ `(7, B)` $\rightarrow$ `(12, E)`.



Collision Resolution 2: Open Addressing

All items are stored directly inside the array. If $h(K)$ is filled, we probe:

1. Linear Probing: $i = (h(K) + j) \bmod M$

Probes slots sequentially.

- **Problem**: **Primary Clustering**. Adjacent filled slots grow into long blocks. Any collision in a cluster prolongs searches and insertions.

2. Quadratic Probing: $i = (h(K) + j^2) \bmod M$

Probes slots quadratically.

- **Benefit**: Eliminates primary clustering.
- **Problem**: Secondary Clustering (keys hashing to same index trace same sequence).

Open Addressing: Double Hashing

Double Hashing eliminates both primary and secondary clustering by determining probe steps with a second hash function $h_2(K)$:

$$\text{Probe Slot } i = (h_1(K) + j \cdot h_2(K)) \bmod M \quad (j = 0, 1, 2, \dots)$$

- The step size depends on the key itself, so different keys hashing to the same slot resolve collisions via different probe paths.
- **Rule for $h_2(K)$:** $h_2(K)$ must never return 0.
- Common formula: $h_2(K) = R - (K \bmod R)$ where $R < M$ is prime.

Example

Let $M = 11$, $h_1(K) = K \bmod 11$, $h_2(K) = 7 - (K \bmod 7)$.

Insert $K = 14$:

- $h_1(14) = 3$. (Slot 3 occupied).
- $h_2(14) = 7 - 0 = 7$.
- Probe 1: $(3 + 7) \bmod 11 = 10$.

Handling Duplicates in Open Addressing

If we use Open Addressing (Linear, Quadratic, or Double Hashing) for a dictionary, how do we insert duplicate keys?

1. **Insertion**: Duplicate key-element is placed in the next available probed slot.
2. **Search (`findAll(k)`)**:
   - We **cannot** stop searching when we find the first occurrence!
   - We must continue probing the sequence until we hit an **empty (never used)** slot.
   - Deleted slots marked with a placeholder (tombstone) do not stop the probe.
3. **Erase**: Must search the entire chain to delete specific values.

Performance Degradation

Because duplicates force searches to scan the entire probe path to the end, Open Addressing performance degrades significantly under high duplication. Separate Chaining or Node Value Lists are preferred.

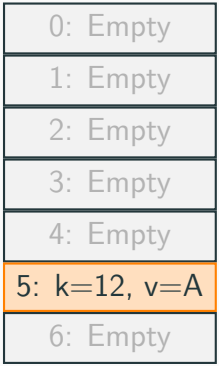
Double Hashing Duplicate Insertion Trace: Step-by-Step

Table size $M = 7$, $h_1(K) = K \bmod 7$, $h_2(K) = 5 - (K \bmod 5)$.

Insert: `(12, A)`, `(5, B)`, `(12, C)` (duplicate key!).

Step-by-step insertion:

- **Step 1**: Insert `(12, A)`.
  $h_1(12) = 12 \bmod 7 = 5$ (empty).
  Insert at slot 5.



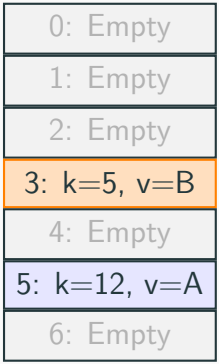
Double Hashing Duplicate Insertion Trace: Step-by-Step

Table size $M = 7$, $h_1(K) = K \bmod 7$, $h_2(K) = 5 - (K \bmod 5)$.

Insert: `(12, A)`, `(5, B)`, `(12, C)` (duplicate key!).

Step-by-step insertion:

- **Step 2**: Insert `(5, B)`.
  $h_1(5) = 5$ (occupied by 12). Collision!
  Step size $h_2(5) = 5 - (5 \bmod 5) = 5$.
  Probe 1: $(5 + 5) \bmod 7 = 3$ (empty).
  Insert at slot 3.



Double Hashing Duplicate Insertion Trace: Step-by-Step

Table size $M = 7$, $h_1(K) = K \bmod 7$, $h_2(K) = 5 - (K \bmod 5)$.

Insert: `(12, A)`, `(5, B)`, `(12, C)` (duplicate key!).

Step-by-step insertion:

- **Step 3**: Insert `(12, C)` (duplicate!).
  $h_1(12) = 5$ (occupied). Collision!
  Step size $h_2(12) = 5 - (12 \bmod 5) = 3$.
  Probe 1: $(5 + 3) \bmod 7 = 1$ (empty).
  Insert at slot 1.

0: Empty
1: k=12, v=C
2: Empty
3: k=5, v=B
4: Empty
5: k=12, v=A
6: Empty

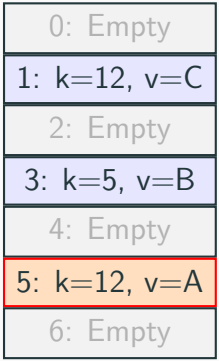
Double Hashing Duplicate Search Trace: Step-by-Step

Search for all values of key 12 (`findAll(12)`) in our Double Hashing table.

$h_1(12) = 5$, step size $h_2(12) = 3$.

Step-by-step search:

- **Initial Probe**: Check slot $h_1(12) = 5$.
  Slot 5 contains key 12 (value `A`).
  Match found! Yield `A`.



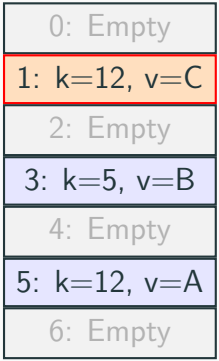
Double Hashing Duplicate Search Trace: Step-by-Step

Search for all values of key 12 (`findAll(12)`) in our Double Hashing table.

$h_1(12) = 5$, step size $h_2(12) = 3$.

Step-by-step search:

- **Probe 1**: Calculate next index:
  $(5 + 3) \bmod 7 = 1$.
  Check slot 1.
  Slot 1 contains key 12 (value `C`).
  Match found! Yield `C`.



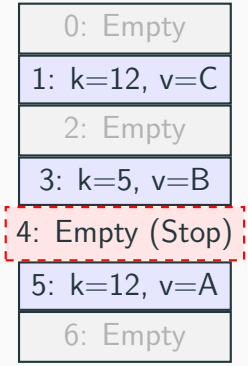
Double Hashing Duplicate Search Trace: Step-by-Step

Search for all values of key 12 (`findAll(12)`) in our Double Hashing table.

$h_1(12) = 5$, step size $h_2(12) = 3$.

Step-by-step search:

- **Probe 2**: Calculate next index:
  $(5 + 2 \cdot 3) \bmod 7 = 4$.
  Check slot 4.
  Slot 4 is **Empty**.
  Stop search! (No more duplicates possible).



Summary of Dictionary Architectures

Comprehensive Implementation Comparison

Let N be the number of items and M be the capacity.

Implementation	Find (Single)	Find All (k)	Insert	Delete
Log File	$\mathcal{O}(N)$	$\mathcal{O}(N)$	$\mathcal{O}(1)$	$\mathcal{O}(N)$
Ordered Array	$\mathcal{O}(\log N)$	$\mathcal{O}(\log N + count)$	$\mathcal{O}(N)$	$\mathcal{O}(N)$
Binary Search Tree	$\mathcal{O}(N)$ worst	$\mathcal{O}(N)$ worst	$\mathcal{O}(N)$ worst	$\mathcal{O}(N)$ worst
Balanced Tree (RBT)	$\mathcal{O}(\log N)$	$\mathcal{O}(\log N + count)$	$\mathcal{O}(\log N)$	$\mathcal{O}(\log N)$
Hash: Chaining	$\mathcal{O}(1 + \lambda)$ avg	$\mathcal{O}(1 + \lambda + count)$ avg	$\mathcal{O}(1)$	$\mathcal{O}(1 + \lambda)$ avg
Hash: Open Address	$\mathcal{O}(\frac{1}{1-\lambda})$ avg	$\mathcal{O}$(Full Probe Chain)	$\mathcal{O}(\frac{1}{1-\lambda})$	$\mathcal{O}(\frac{1}{1-\lambda})$

- **Tree-based** guarantees stable sub-linear bounds, keeping duplicate values together inside nodes or adjacent subtrees.
- **Hash Chaining** scales gracefully with duplicates by appending to local chains.
- **Open Addressing** should be avoided if duplicate keys or deletions are frequent.